

Dynamic Authorization & Dynamic Menu — Dokumen Belajar Lengkap

Dokumen ini isinya semua yang udah kita kerjakan dari nol, lengkap sama alasan kenapa tiap langkah dilakukan dan penjelasan detail tiap kode. Tujuannya biar kamu paham konsepnya, bukan cuma hafal langkahnya.

DAFTAR ISI

1. Konsep Dasar (wajib paham duluan)
 2. Setup Awal Project
 3. Implementasi Tiap Poin SOP
 4. Penjelasan Detail Semua Kode
 5. Ringkasan Alur Kerja Sistem
 6. Kemungkinan Pertanyaan Pembimbing + Jawaban
-

1. KONSEP DASAR

1.1 Role vs Permission

Ini konsep paling fundamental, harus paham duluan sebelum lanjut ke yang lain.

- **Role** = “jabatan”/“peran” seseorang. Contoh: admin, staff.
- **Permission** = “izin buat ngelakuin satu hal spesifik”. Contoh: `users.read`, `users.create`.

Analogi kantor: Role itu kayak jabatan “Manager”. Permission itu kayak akses spesifik: “boleh buka ruang server”, “boleh approve cuti”. Jabatan Manager itu isinya **kumpulan** dari beberapa izin akses tadi.

Kenapa dipisah, gak digabung aja? Karena kalau digabung, sistemnya jadi kaku. Bayangin kalau cuma ada “role” tanpa “permission” — berarti tiap kali mau ngatur “staff boleh lihat data tapi gak boleh hapus”, kamu harus bikin role baru khusus, misal `staff_readonly`. Lama-lama role-nya bejibun. Dengan sistem permission terpisah, 1 role bisa punya kombinasi izin yang fleksibel tanpa perlu bikin role baru tiap ada kebutuhan baru.

1.2 Kenapa harus “dinamis” (dari database), bukan hardcoded?

Hardcode artinya nulis logic langsung di kode program, contoh:

```
if ($user->name === 'Reva') {  
    tampilkan_menu_admin();  
}
```

Ini jelek karena: kalau mau tambah admin baru atau ubah menu, kamu harus **edit kode dan deploy ulang**. Ribet, rawan salah, dan gak scalable.

Sistem dinamis artinya semua aturan (siapa boleh apa, menu apa aja yang ada) **disimpan sebagai data** di database. Untuk ubah aturan, tinggal ubah data — gak perlu sentuh kode program sama sekali.

1.3 Kenapa pakai Laravel + Spatie, bukan bikin sendiri dari nol?

Sistem role & permission itu **rumit** kalau dibikin sendiri dari nol — ada banyak edge case (1 user bisa punya banyak role, 1 role bisa punya banyak permission, dll). Spatie laravel-permission itu package yang sudah dipakai dan diuji oleh jutaan developer Laravel di seluruh dunia. Daripada reinvent the wheel, kita “numpang” logic yang udah teruji itu.

2. SETUP AWAL PROJECT

Langkah 1 — Cek tools yang dibutuhkan

Kita cek dulu PHP, Composer, dan database (Postgres) sudah terpasang. Ini prasyarat mutlak — tanpa ini, Laravel gak akan bisa jalan sama sekali.

Langkah 2 — Bikin project Laravel baru

```
composer create-project laravel/laravel dynamic-menu-app
```

Apa yang terjadi: Composer mendownload seluruh framework Laravel (struktur folder, dependency dasar) dan naruhnya di folder dynamic-menu-app. Ini kayak dapat “cetakan kosong” aplikasi web yang siap dikembangkan.

Langkah 3 — Bikin database kosong dynamic_menu_db di PostgreSQL

Kita bikin lewat pgAdmin, taruh di server PostgreSQL 17 yang udah ada (gak perlu server/instalasi Postgres baru — cukup satu instalasi bisa nampung banyak database sekaligus, sama kayak 1 lemari bisa punya banyak laci).

Langkah 4 — Setting .env

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=dynamic_menu_db
DB_USERNAME=postgres
DB_PASSWORD=(password kamu)
```

Kenapa file ini penting: .env itu “kartu identitas” konfigurasi aplikasi kamu. Laravel baca file ini buat tau harus connect ke database mana, pakai kredensial apa. Tanpa ini diisi benar, Laravel gak akan bisa komunikasi ke Postgres sama sekali.

Penjelasan tiap baris: - DB_CONNECTION=pgsql → pakai “bahasa”/driver Postgres (defaultnya Laravel disetel ke mysql) - DB_HOST=127.0.0.1 → server database ada di mesin yang sama (laptop kamu sendiri) - DB_PORT=5432 → nomor “pintu masuk” khusus Postgres (beda dari MySQL yang di 3306) - DB_DATABASE → nama database yang dituju - DB_USERNAME / DB_PASSWORD → kredensial login, sama kayak login ke pgAdmin

Langkah 5 — Aktifkan extension pdo_pgsql di PHP

PHP butuh “penerjemah” khusus buat bisa “ngobrol” ke tiap jenis database. Extension pdo_pgsql itu penerjemahnya buat Postgres. Kita aktifkan di php.ini (hapus tanda ; di depan baris extension=pdo_pgsql dan extension=pgsql).

Analogi: .env itu alamat rumah tujuan, pdo_pgsql itu SIM buat bisa nyetir ke sana. Alamat benar tapi gak punya SIM ya tetap gak nyampe.

Langkah 6 — php artisan migrate

Migration adalah cara Laravel bikin struktur tabel database **lewat kode PHP**, bukan lewat SQL manual. Command ini menjalankan semua file migration yang ada, salah satunya bikin tabel bawaan Laravel: users, sessions, cache, jobs, dll.

Kenapa pakai migration, bukan bikin tabel manual di pgAdmin? Karena migration itu **tercatat sebagai kode** (versioned). Kalau kerja tim atau pindah laptop, tinggal jalankan migrate, struktur database otomatis terbentuk identik — gak perlu ingat-ingat query SQL manual satu-satu. Ini juga alasan di balik poin 8 SOP (“berbasis database, tanpa hardcode”) — struktur/logic didefinisikan di kode migration, bukan diketik ulang manual tiap saat.

3. IMPLEMENTASI TIAP POIN SOP

POIN 1 — Spatie Laravel Permission sebagai dasar

```
composer require spatie/laravel-permission
php artisan vendor:publish --
provider="Spatie\Permission\PermissionServiceProvider"
php artisan migrate
```

Apa yang terjadi tiap command: 1. composer require → download package Spatie, daftarkan di composer.json 2. vendor:publish → menyalin file migration & config dari dalam folder vendor/ (folder “tersembunyi” isi semua package) ke folder project kamu (database/migrations/, config/permission.php) — supaya bisa kamu lihat/edit/jalankan 3. migrate → benar-benar mengeksekusi migration itu, bikin 5 tabel: roles, permissions, model_has_roles, model_has_permissions, role_has_permissions

Kenapa ada tabel model_has_roles, bukan langsung kolom di tabel users? Karena Spatie didesain generic/fleksibel — bukan cuma model User yang bisa dikasih role, model lain (misal Admin, Vendor) di masa depan juga bisa pakai sistem yang sama. Makanya namanya model_has_roles (general), bukan user_has_roles (spesifik).

Tambahan wajib: trait HasRoles ditambahkan ke app/Models/User.php:

```
use Spatie\Permission\Traits\HasRoles;

class User extends Authenticatable
{
    use HasFactory, Notifiable, HasRoles;
}
```

Kenapa ini wajib? Tanpa trait ini, model User **gak “kenal”** method-method dari Spatie seperti assignRole(), hasRole(), can(). Trait itu yang “menyuntikkan” kemampuan tersebut ke dalam model. Ini pernah kejadian di project kita — waktu lupa nambahin trait ini, muncul error Call to undefined method assignRole().

POIN 2 — Tabel menu dinamis (menu + sub menu)

```
Schema::create('menus', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('slug')->unique();
    $table->string('permission_prefix')->unique();
    $table->string('icon')->nullable();
    $table->string('route')->nullable();
    $table->unsignedBigInteger('parent_id')->nullable();
    $table->integer('order')->default(0);
    $table->boolean('is_active')->default(true);
    $table->timestamps();

    $table->foreign('parent_id')->references('id')->on('menus')->onDelete('cascade');
});
```

Penjelasan tiap kolom: - name → nama yang ditampilkan ke user, misal “Manajemen User” - slug → identitas unik dalam bentuk teks pendek, misal “users” (biasa dipakai di URL) - permission_prefix → basis pembentukan nama permission (dijelaskan di Poin 3) - icon, route → data pendukung tampilan (opsional, boleh kosong/nullable) - parent_id → **ini kunci dari fitur sub menu** - order → urutan tampil di sidebar - is_active → biar bisa “matikan” menu tanpa harus hapus datanya

Kenapa parent_id yang dipakai buat sub menu, bukan bikin tabel submenus terpisah? Teknik ini namanya **self-referencing table** — 1 tabel menus bisa berperan sebagai menu utama ATAU sub menu, tergantung isi parent_id: - parent_id = null → ini menu utama (tidak “menempel” ke menu manapun) - parent_id = 2 → ini sub menu dari menu yang ber-id 2

Keuntungannya: tidak perlu tabel terpisah, dan secara teori bisa mendukung sub-sub-menu (menu di dalam sub menu) kalau suatu saat dibutuhkan.

Kenapa slug dan permission_prefix harus unique()? - slug unik → mencegah 2 menu punya identitas yang sama/bentrok - permission_prefix unik → mencegah 2 menu menghasilkan nama permission yang sama. Kalau dua menu sama-sama pakai prefix users, permission users.read jadi ambigu — milik menu yang mana?

\$table->foreign('parent_id')->...->onDelete('cascade'); ini artinya kalau menu induk dihapus, semua sub menunya ikut terhapus otomatis (biar tidak ada sub menu yang “menggantung” tanpa induk).

POIN 3 — Satu menu, banyak permission (permission_prefix.action)

Di app/Models/Menu.php:

```
public function permissionNames(): array
{
    return collect(['create', 'read', 'update', 'delete'])
        ->map(fn ($action) => "{$this->permission_prefix}.$action")
        ->toArray();
}
```

Cara kerja: kalau permission_prefix menu itu isinya "users", method ini otomatis menghasilkan array:

```
["users.create", "users.read", "users.update", "users.delete"]
```

Kenapa harus 4 action ini (CRUD)? CRUD (Create, Read, Update, Delete) adalah konsep standar dalam pengembangan software — hampir semua operasi terhadap data bisa dipetakan ke 4 hal ini: bikin baru, lihat, ubah, hapus.

Lalu di app/Services/MenuPermissionService.php:

```
public function generatePermissionsForMenu(Menu $menu): void
{
    foreach ($menu->permissionNames() as $permissionName) {
        Permission::firstOrCreate([
            'name' => $permissionName,
            'guard_name' => 'web',
        ]);
    }
}
```

Penjelasan: method ini mengambil 4 nama permission dari `permissionNames()`, lalu untuk masing-masing, cek ke tabel `permissions` — kalau belum ada baris dengan nama itu, buat baru (`firstOrCreate`). Kalau sudah ada, dilewati saja (tidak duplikat).

Kenapa gak nulis 4 permission itu manual satu-satu tiap kali bikin menu baru? Karena otomatisasi ini mengurangi human error (lupa bikin salah satu permission) dan konsisten — pola penamaannya selalu sama, `{prefix}.{action}`, tidak akan ada yang beda-beda gaya penulisan.

POIN 4 — Tabel pivot menu_role

```
Schema::create('menu_role', function (Blueprint $table) {
    $table->id();
    $table->foreignId('menu_id')->constrained('menus')-
>onDelete('cascade');
    $table->foreignId('role_id')->constrained('roles')-
>onDelete('cascade');
    $table->timestamps();

    $table->unique(['menu_id', 'role_id']);
});
```

Kenapa butuh tabel pivot terpisah, gak taruh role_id langsung di tabel menus? Karena relasinya bersifat **many-to-many** (banyak ke banyak): - 1 menu bisa dilihat oleh **banyak** role (contoh: menu “Pembayaran” bisa dilihat admin DAN staff) - 1 role bisa melihat **banyak** menu (contoh: admin bisa lihat semua menu)

Kalau `role_id` ditaruh langsung sebagai kolom di tabel `menus`, itu cuma bisa menampung **1 role per menu** (relasi many-to-one) — tidak cukup untuk kasus di atas. Tabel pivot menyelesaikan ini: tiap baris cuma menyimpan 1 pasangan (`menu_id`, `role_id`). Kalau ada 2 role yang boleh lihat 1 menu, tinggal ada 2 baris di tabel pivot.

->constrained('menus') dan ->constrained('roles'): ini foreign key constraint — memastikan `menu_id` dan `role_id` yang dimasukkan **wajib** menunjuk ke data yang benar-benar ada di tabel `menus` dan `roles`. Postgres otomatis menolak kalau ada yang mencoba masukkan id yang tidak eksis. Ini menjaga integritas data.

->unique(['menu_id', 'role_id']): mencegah kombinasi `menu+role` yang sama terdaftar dua kali (misal admin + menu users tidak bisa muncul 2 baris identik).

Relasi di Model Menu.php:

```

public function roles(): BelongsToMany
{
    return $this->belongsToMany(Role::class, 'menu_role');
}

```

Ini yang membuat kita bisa menulis \$menu->roles (ambil semua role yang boleh lihat menu ini) atau \$menu->roles()->sync([...]) (atur role mana saja yang boleh lihat menu ini).

POIN 5 & 6 — Role atur tampilan menu, Permission atur aksi, Menu tampil dinamis

Ini bagian paling penting secara konsep. Logic-nya ada di buildMenuForUser():

```

public function buildMenuForUser($user = null): array
{
    $user = $user ?? Auth::user();

    if (! $user) {
        return [];
    }

    $userRoleIds = $user->roles->pluck('id');

    $menus = Menu::parents()
        ->whereHas('roles', fn ($q) => $q->whereIn('roles.id',
$userRoleIds))
        ->with(['children' => function ($q) use ($userRoleIds) {
            $q->whereHas('roles', fn ($q2) => $q2->
>whereIn('roles.id', $userRoleIds));
        }])
        ->get();

    return $menus
        ->filter(fn ($menu) => $user->can("{ $menu->
>permission_prefix}.read"))
        ->map(function ($menu) use ($user) {
            $menu->setRelation(
                'children',
                $menu->children->filter(fn ($child) => $user->can("
{$child->permission_prefix}.read"))->values()
            );
            return $menu;
        })
        ->values()
        ->toArray();
}

```

Dibedah baris per baris:

1. \$user = \$user ?? Auth::user(); → kalau tidak dikasih user secara eksplisit, ambil user yang sedang login. Auth::user() itu cara Laravel tau “siapa yang lagi login sekarang” (berdasarkan session).
2. if (! \$user) return []; → kalau tidak ada yang login, kembalikan array kosong (tidak ada menu yang ditampilkan). Ini pengaman dasar.
3. \$userRoleIds = \$user->roles->pluck('id'); → ambil semua ID role yang dimiliki user itu. Contoh: kalau user itu punya role admin (id=1), maka \$userRoleIds isinya [1].

4. `Menu::parents()` → ambil menu utama saja (bukan sub menu), yang aktif. Ini scope yang kita definisikan sendiri di Model.
5. `->whereHas('roles', fn ($q) => $q->whereIn('roles.id', $userRoleIds))` → **LAPISAN PERTAMA**: filter, cuma ambil menu yang di tabel `menu_role`-nya terhubung ke salah satu role yang dimiliki user ini. Ini yang mewujudkan **“Role atur tampilan menu”** (Poin 5).
6. `->with(['children' => ...])` → sekaligus ambil sub menu (`children`), dengan filter role yang sama.
7. `->filter(fn ($menu) => $user->can("{ $menu->permission_prefix}.read"))` → **LAPISAN KEDUA**: dari menu yang lolos filter role tadi, cek lagi — apakah user ini (lewat role-nya) benar-benar punya permission `{prefix}.read`? Kalau tidak, menu itu dibuang dari hasil akhir. Ini mewujudkan **“menu tampil hanya jika permission read tersedia”** (Poin 6).
8. Bagian `map()` di bawahnya melakukan hal yang sama untuk `children` (sub menu) — filter ulang berdasarkan permission `read`.

Kenapa 2 lapis filter (role DAN permission), padahal kelihatannya mirip/redundant?

Ini disengaja untuk pemisahan tanggung jawab yang jelas: - `menu_role` (lapis 1) menjawab pertanyaan: **“menu ini ditujukan untuk role siapa saja?”** - Permission `read` (lapis 2) menjawab pertanyaan: **“apakah role itu benar-benar sedang diizinkan untuk membaca/melihat isi menu ini?”**

Manfaatnya: kalau suatu saat ingin mencabut akses baca dari suatu role tanpa harus menghapus relasi `menu_role`-nya (misal role itu nonaktif sementara), cukup cabut permission `read`-nya saja, tidak perlu ubah struktur relasi menu.

Contoh nyata dari project kita: role `staff` terhubung ke menu “Pembayaran” di tabel `menu_role` (lolos lapis 1), DAN role `staff` juga diberi permission `payments.read` (lolos lapis 2) — jadi menu itu **muncul** untuk `staff`. Tapi `staff` **tidak** diberi `payments.create`, `payments.update`, atau `payments.delete` — jadi walau menunya kelihatan, dia tidak bisa melakukan aksi CRUD selain melihat data.

POIN 7 — Route dilindungi middleware

```
Route::get('/users-only', function () {
    return 'Kamu berhasil akses halaman Manajemen User!';
})->middleware(['auth', 'permission:users.read']);
```

Kenapa menyembunyikan menu di tampilan saja tidak cukup?

Karena menyembunyikan menu itu **hanya soal tampilan (frontend)**. Kalau seseorang tahu alamat URL-nya secara langsung (misal `/users-only`), dia tetap bisa mengetiknya manual di address bar dan mengakses halaman itu — **kecuali** ada penjagaan di level server (backend) yang benar-benar mengecek izin setiap kali ada permintaan (request) masuk, apapun jalurnya (klik menu ataupun ketik URL langsung).

Cara kerja middleware `permission:users.read`: Middleware itu seperti “satpam” yang berdiri di depan pintu (route). Setiap request yang mau masuk, dicek dulu oleh satpam ini: “apakah user yang login ini punya izin `users.read`?” Kalau tidak, request langsung ditolak dengan response **403 Forbidden**, kode di dalam route (fungsi closure-nya) bahkan tidak akan pernah dijalankan sama sekali.

Bukti nyata yang sudah kita uji coba: user Staff Test (role staff, yang cuma punya permission payments.read & payment_history.read, TIDAK punya users.read) mencoba akses /users-only secara langsung lewat URL — hasilnya **403 Forbidden**, walaupun dia sudah login. Ini membuktikan proteksi bekerja di level backend, bukan cuma menyembunyikan tombol di frontend.

Kenapa perlu daftarkan alias middleware di bootstrap/app.php?

```
->withMiddleware(function (Middleware $middleware): void {
    $middleware->alias([
        'role' =>
\Spatie\Permission\Middleware\RoleMiddleware::class,
        'permission' =>
\Spatie\Permission\Middleware\PermissionMiddleware::class,
        'role_or_permission' =>
\Spatie\Permission\Middleware\RoleOrPermissionMiddleware::class,
    ]);
})
```

Laravel butuh tahu: kalau ada yang menulis middleware('permission:...') di route, itu maksudnya memanggil class PHP yang mana. Baris alias ini yang menjadi jembatan/kamus terjemahan antara nama pendek (permission) dengan nama class lengkapnya (Spatie\Permission\Middleware\PermissionMiddleware).

POIN 8 — Semua berbasis database, tanpa hardcoded

Ini bukan 1 baris kode spesifik, melainkan **prinsip yang dipegang di seluruh sistem** yang sudah kita bangun:

Yang mungkin di-hardcode	Yang kita lakukan (berbasis database)
if (\$user->name == 'Reva') ...	Cek lewat \$user->hasRole('admin') — role disimpan di tabel
Daftar menu ditulis manual di file Blade/HTML	Menu diambil dari tabel menus, dirender secara dinamis
Daftar permission ditulis manual di kode	Permission dihasilkan otomatis dari permission_prefix di database
Role mana lihat menu mana ditulis di kode (if role == admin show menu)	Disimpan di tabel menu_role, tinggal query

Kenapa ini penting? Karena untuk mengubah “siapa boleh lihat menu apa” atau “menambah menu baru”, cukup ubah **data** (lewat form/seeders/CRUD), tidak perlu mengubah dan men-deploy ulang kode program. Ini yang membuat sistem disebut benar-benar “dinamis”.

4. PENJELASAN FILE-FILE PENDUKUNG LAIN

MenuController.php (contoh CRUD menu)

```
public function __construct(protected MenuPermissionService
$menuPermissionService)
{
    $this->middleware('permission:menus.read')->only(['index',
```

```
'show']));
    $this->middleware('permission:menus.create')->only(['store']);
    $this->middleware('permission:menus.update')->only(['update']);
    $this->middleware('permission:menus.delete')->only(['destroy']);
}
```

Penjelasan: ini contoh bagaimana 1 controller bisa punya proteksi permission **berbeda-beda per fungsi (method)**. Fungsi store() (bikin data baru) butuh permission menus.create, sedangkan destroy() (hapus data) butuh menus.delete. ->only([...]) membatasi middleware itu cuma berlaku untuk method yang disebutkan.

Di dalam store():

```
$menu = Menu::create($validated);
$this->menuPermissionService->generatePermissionsForMenu($menu);

if (! empty($validated['role_ids'])) {
    $menu->roles()->sync($validated['role_ids']);
}
```

Alur ini penting dipahami — setiap kali admin membuat menu baru lewat form: 1. Data menu disimpan (Menu::create) 2. Otomatis dibuatkan 4 permission untuknya (generatePermissionsForMenu) 3. Role yang dipilih di-assign ke menu itu lewat tabel pivot (roles()->sync())

Ini rangkaian lengkap yang mewujudkan poin 2, 3, dan 4 SOP sekaligus dalam satu alur kerja.

MenuPermissionSeeder.php

Perlu diingat: **seeder bukan poin yang diminta di SOP**, ini murni alat bantu kita untuk membuat data contoh secara cepat (role admin/staff, beberapa menu) supaya logic yang sudah dibangun bisa langsung dibuktikan bekerja, tanpa harus input data manual satu per satu lewat pgAdmin.

5. RINGKASAN ALUR KERJA SISTEM (End-to-End)

Begini alur lengkapnya kalau digambarkan sebagai cerita:

1. **Admin bikin menu baru** lewat form (MenuController@store) → data masuk ke tabel menus
2. **Sistem otomatis membuat 4 permission** untuk menu itu ({prefix}.create/read/update/delete) → masuk ke tabel permissions
3. **Admin pilih role mana yang boleh lihat menu itu** → tersimpan ke tabel pivot menu_role
4. **Admin assign permission yang relevan ke role** (misal: role staff cuma dikasih {prefix}.read) → tersimpan ke tabel role_has_permissions
5. **User login** → sistem tahu role apa saja yang dimiliki user itu (dari model_has_roles)
6. **Saat user buka dashboard**, sistem panggil buildMenuForUser():
 - Cari menu yang role-nya cocok dengan role user (cek menu_role)
 - Dari situ, saring lagi — hanya tampilkan yang permission read-nya dimiliki user
7. **User klik/akses menu tertentu** → request masuk ke route yang dituju
8. **Middleware permission:...** mengecek ulang di level server — apakah user benar-benar punya izin untuk aksi itu (create/read/update/delete)? Kalau tidak, ditolak dengan 403, walau user sudah berhasil login

6. KEMUNGKINAN PERTANYAAN PEMBIMBING + JAWABAN SINGKAT

Q: Kenapa pakai Spatie, bukan bikin sistem role/permission sendiri? A: Karena sistem role & permission itu kompleks (banyak relasi many-to-many), Spatie sudah teruji dan dipakai luas oleh komunitas Laravel, jadi lebih efisien dan minim bug dibanding membuat dari nol.

Q: Apa bedanya role dan permission dalam sistem ini? A: Role menentukan menu apa yang tampil (lewat tabel `menu_role`), sedangkan permission menentukan aksi apa yang boleh dilakukan di dalam menu itu (`create/read/update/delete`).

Q: Kenapa menu bisa punya sub menu tanpa tabel terpisah? A: Karena pakai teknik self-referencing table — kolom `parent_id` di tabel `menus` menunjuk ke `id` menu lain. Kalau `null`, berarti dia menu utama; kalau terisi, berarti dia sub menu dari menu tersebut.

Q: Kenapa 1 menu bisa punya banyak permission? A: Karena setiap menu punya `permission_prefix`, dan sistem otomatis membuat 4 permission dari kombinasi `{prefix}.create`, `{prefix}.read`, `{prefix}.update`, `{prefix}.delete` — supaya setiap aksi CRUD di menu itu bisa diatur izinnya secara terpisah.

Q: Bagaimana cara memastikan orang tidak bisa akses langsung lewat URL kalau menunya disembunyikan? A: Setiap route dilindungi middleware `permission`: dari Spatie. Middleware ini mengecek izin di level server sebelum request diproses — jadi walau seseorang tahu URL-nya dan mengetik langsung, tetap akan ditolak (403 Forbidden) kalau dia tidak punya permission yang sesuai.

Q: Apa buktinya sistem ini “berbasis database”, bukan hardcoded? A: Semua menu, role, permission, dan relasi antar-role-menu disimpan sebagai baris data di tabel database (`menus`, `roles`, `permissions`, `menu_role`, `role_has_permissions`). Untuk menambah menu baru atau mengubah siapa yang boleh mengakses apa, cukup ubah data lewat form/seeder — tidak perlu mengubah kode program.